Appendix

This appendix supplements the main paper. Section S1 gives the construction details of AmbiOverpass-Bench summarized in Section III of the main paper. Section S2 shows benchmark instances, and Section S3 reports the quality control. Section S4 proves Theorems 1 and 2 and the properties of the coalition values, the Shapley values and the superset execution used in Section IV. Section S5 lists the experimental settings, Section S6 reports additional results for Section V, and Section S7 lists all prompts. Equations, algorithms, definitions, figures and tables without the prefix S refer to the main paper.

S1. AmbiOverpass-Bench

AmbiOverpass-Bench evaluates interactive disambiguation for Text-to-OverpassQL. It starts from the executable request–query pairs of OverpassNL [1], turns each precise request into an ambiguous one, and keeps the reference query as the intended interpretation together with its execution result. The benchmark has three planning subsets with an increasing number of interacting slots and one rewriting subset for candidate generation. Section III-A and Fig. 2 of the main paper outline the construction, and this section gives the rules and settings of each step.

A. Construction Pipeline

a) Source query analysis: The first step finds the slots that a request can leave open. We parse each OverpassNL reference query and extract its area, target tag, element type, additional conditions and proximity constraints. The reference query is the intended interpretation of the request. We keep queries that return objects and whose structure contains the slots listed above.

b) Ambiguity injection: The second step turns a precise request into an ambiguous one with a known intended reading. For each source query, we select a subset of its slots and collect candidate values for them. Area candidates are the OSM areas with the same name. Tag candidates are checked against global OSM tag statistics. Element types and conditions are varied according to the query structure, and landmark requests also vary the landmark tag and the radius. Because every candidate value comes from OSM, every reading is an executable query. Qwen3-14B then rewrites the precise request so that the selected slots become vague and the other slots stay as specific as before. The intended reading is therefore always among the candidates, and the other readings differ from it only in the weakened slots. Automatic checks reject rewrites that contain tag syntax, geometry terms or the original value of a weakened slot.

c) Execution-based annotation: The third step records what every reading returns. For each ambiguous request, we enumerate the combinations of candidate values. Each combination is one reading of the request, that is, one candidate interpretation h in the notation of the main paper. Superset execution (Section IV-B of the main paper) executes all readings together. Readings with empty results are removed, and readings that return the same objects form one result class. We keep instances in which the intended reading returns objects and at least two slots change the result. Each instance stores the precise request, the ambiguous request, the candidate values, the intended values, the candidate queries, the cached results and the result classes. Since all planners read the same cached results and result classes, differences between planners come from which questions they ask and when they stop.

B. Subsets

a) Single-object requests: This subset tests question selection over the basic slots of a request. These requests ask for one kind of object. Two to four of the slots (area, target tag, element type and additional condition) are ambiguous. An area name matches up to four OSM areas, and the element type is one of `node`, `way`, `relation` and `nwr`. When a condition is left out of the request, we add a twin instance with the same ambiguous request and the opposite intended value of the condition slot. A fixed choice for the condition is then wrong on one instance of each pair.

b) Single-landmark requests: This subset adds a proximity constraint, whose landmark tag and radius interact with the target tag. These requests ask for objects near a landmark. The landmark is a station, school, park, hospital or supermarket, and each kind has two or three candidate OSM tags. The radius is 100, 200 or 500 meters. The intended landmark tag and radius are drawn from the combinations with non-empty results, so the intended value does not sit at a fixed option position.

c) Dual-landmark requests: This subset tests planning when many slots interact. These requests ask for objects near two landmarks at the same time. Each landmark adds a tag slot and a radius slot, so together with the area, target tag and element type a request has up to seven slots. A change of one landmark or radius can change whether another slot still affects the result.

d) Rewriting requests: These requests test candidate generation from indirect descriptions. The rewrites use everyday words for the object, describe it through properties that OSM does not record, add requirements that the database cannot check, or give the place by its bare name. For example, a

request for pharmacies becomes a request for "five-star health stores". The intended tag and query are kept as references.

C. Splits and Evaluation Protocol

The splits keep the test data out of every design decision. All instances derived from one source query are placed in the same split. The prior calibration and all method settings are chosen on the development split. The test split and the held-out splits are used only for evaluation. The protocol evaluates planning apart from candidate generation. The planning experiments use a simulated user whose intent is the intended interpretation. Asked about a slot, the user returns the intended value, or "no preference" with probability ε. All planners receive the same candidate interpretations, result classes, prior and budget, and none of them sees the intended values. The planning evaluation therefore measures question selection and stopping on given candidates. Candidate generation is evaluated separately by the coverage of the intended tag on the rewriting subset. We also label each planning instance by the dependency structure of its slots, which describes how strongly the slots interact. In factorisable instances the slots can be resolved independently. In gated-forest instances the answer to one slot can remove other slots. All remaining instances are labeled general.

S2. Benchmark Examples

Table S1 shows one test instance from each planning subset, and Table S2 shows three rewriting instances.

The planning instances show how the slots interact in each subset. The two single-object instances share the ambiguous request "Where are the toll stations in Brazil?" and differ only in the intended value of the condition slot, so a fixed answer for this slot is wrong on one of them. The single-landmark instance has four candidate areas, two target tags, three landmark tags, three radii and four element types. Its 30 non-empty readings form 18 result classes. The dual-landmark instance has a fixed area and six ambiguous slots. Its 131 non-empty readings form only 13 result classes, because most slot changes return the same schools. A question helps this instance only when its answer separates these 13 classes, and a question that only separates readings within one class uses budget without changing the submission. Result classes (Definition 2 of the main paper) remove such questions.

Table S2 shows why the rewriting subset needs more than direct tag generation. Each request names the object in everyday words or through a property that OSM does not record. In all three instances, a single list of 50 LLM-proposed tags misses the intended tag. For "the best places for swimming", the list does not contain `natural=water`. Correction turns this request into four requests about recordable kinds of objects, and tag alignment of these four requests finds `natural=water`. For the Santa Maria request, relaxation also removes the requirements "quiet" and "well-reviewed", which the database cannot check.

S3. Benchmark Quality Control

Quality control removes generated requests that reveal the intended values or lose a valid intended reading. Table S3 lists how many requests each subset generates and removes. For single-object requests, Qwen3-14B writes 820 ambiguous requests from 187 source queries, and the automatic checks remove 85 of them. Of the removed requests, 81 still name the intended tag value, 1 repeats the precise request, and 3 do both. Adding the 241 twin instances gives 976 instances. For the landmark subsets, an instance is removed when its intended reading returns no object under exact proximity evaluation or when the area lookup fails. This removes 4 single-landmark and 10 dual-landmark instances. For the rewriting subset, three checks remove 37 of the 291 rewrites. The automatic checks remove 18 rewrites that keep the original object word. An LLM judge that knows the intended object removes 15 rewrites from which the object cannot be inferred. The remaining 4 removed rewrites lose the place name. This leaves 254 instances.

S4. Proofs

This section proves the two theorems of the main paper and the properties of the coalition values, the Shapley values and the superset execution that AmbiOverpass uses. S denotes a clarification state, probabilities are taken under $p(h \mid S)$, and a submission returns an interpretation of the most probable result class.

A. Hardness of Clarification Planning

Theorem 1 of the main paper shows that optimal clarification planning is intractable, which motivates the approximate planner of AmbiOverpass.

Theorem 1 (Hardness). *With $\varepsilon = 0$ and an unlimited question budget, finding a clarification policy that identifies $c(h^*)$ with the minimum expected number of questions is NP-hard.*

Proof. The reduction maps each binary test to a slot and each object to an interpretation with its own result class. We reduce from the minimum expected-depth decision tree problem, which is NP-hard [2]. An instance has objects $x_1, \ldots, x_n$ and binary tests $\tau_1, \ldots, \tau_m$ with $\tau_j(x_i) \in \{0, 1\}$, and any two objects differ in at least one test. The goal is a decision tree that identifies every object with the minimum average depth. We build a clarification instance with slots $v_1, \ldots, v_m$, $C_{v_j} = \{0, 1\}$ and no slot dependencies, over a snapshot with objects $o_1, \ldots, o_n$. For each x_i, the interpretation h^i has $h^i_{v_j} = \tau_j(x_i)$ and $R(h^i) = \{o_i\}$, and every other value combination returns no object. Then $\mathcal{H} = \{h^1, \ldots, h^n\}$, each interpretation is its own result class, and the prior is uniform. The construction takes polynomial time.

With $\varepsilon = 0$, a question about v_j in state S returns $\tau_j(x^*)$ and leads to $S_{v_j = \tau_j(x^*)}$. Each result class has one interpretation, so a policy identifies $c(h^*)$ exactly when it submits from a state with $|S| = 1$. Such a policy is a decision tree over the tests whose leaves are the singleton states, and the number of questions for h^i is the depth of x_i. Every decision tree also defines a policy with the same number of questions for each object. A question about a slot that does not split S leaves S unchanged, and removing it does not increase the number

Table S1
PLANNING INSTANCES FROM THE TEST SPLITS, WITH INTENDED VALUES IN BOLD.

	Single-object (twin pair)	Single-landmark	Dual-landmark
Precise request	Where are the toll booths that require a charge in Brazil?	What pharmacies are within 500 metres of a park in Malmö?	Schools in Győr within 500 metres of a park and within 500 metres of a train station.
Ambiguous request	Where are the toll stations in Brazil?	What pharmacies are near a park in Malmö?	Schools in Győr near a park and close to a train station.
Area	four administrative relations named Brasil (**the country**)	**Malmö municipality**, two areas named Malmö, one islet	Győr (single candidate)
Target tag	**`barrier=toll_booth`**, `toll=yes`	**`amenity=pharmacy`**, `shop=chemist`	**`amenity=school`**, `amenity=kindergarten`
Element type	**`nwr`**, `node`, `way`, `relation`	**`nwr`**, `node`, `way`, `relation`	**`nwr`**, `node`, `way`, `relation`
Condition	keep `charge` (**instance A**), drop `charge` (**instance B**)	–	–
Landmark 1	–	`leisure=park`, `landuse=recreation_ground`, **`leisure=garden`**	`leisure=park`, **`landuse=recreation_ground`**, `leisure=garden`
Landmark 2	–	–	**`railway=station`**, `public_transport=station`, `amenity=bus_station`
Radius	–	100 m, 200 m, **500 m**	100 m, 200 m, **500 m** for each landmark
Readings / classes	11 / 10	30 / 18	131 / 13

Table S2
REWRITING INSTANCES AND THE PRECISE REQUESTS PRODUCED BY THE REWRITING OPERATIONS.

Precise request (intended tag)	Rewritten request	Perturbations	Precise requests produced by rewriting
What are the bus routes in Vendée? (`route=bus`)	What are the public transport lines in Vendée?	everyday wording, bare place name	What are the bus routes in Vendée? What are the train lines in Vendée? What are the tram routes in Vendée? What are the ferry routes in Vendée?
Show all water bodies in Curaçao. (`natural=water`)	Show all the best places for swimming in Curaçao.	unrecorded property, uncheckable requirement	Show all swimming pools in Curaçao. Show all beaches in Curaçao. Show all public parks in Curaçao. Show all recreational facilities in Curaçao.
What public transport platforms are in Santa Maria, RS, Brazil, excluding bus stops? (`public_transport=platform`)	What are the quiet, well-reviewed places in Santa Maria that serve as hubs for public transport, excluding bus stops?	unrecorded property, bare place name, uncheckable requirement	Well-reviewed train stations / transit centers / public transport hubs / transportation terminals in Santa Maria that serve public transport.

Table S3
CONSTRUCTION STATISTICS OF AMBIOVERPASS-BENCH.

Subset	Sources	Generated	Removed	Final
Single-object	187	820	85	**976***
Single-landmark	116	347	4	**343**
Dual-landmark	93	445	10	**435**
Rewriting	291	291	37	**254**

*735 retained requests and 241 twin instances.

of questions. The minimum expected number of questions therefore equals the minimum average depth.

The reduction takes the results and classes as given, so the planning problem is NP-hard even after execution grounding.

B. Coalition Values and Shapley Values

AmbiOverpass ranks questions by the Shapley values of the coalition values. The results below show that these values are non-negative, sum to the joint gain of all open slots, and lead to the stopping behavior of Section IV-C of the main paper.

Let $g_S(U) = \sum_x \max_c \Pr(h_U = x,\, c(h) = c \mid S)$. Then $\Delta_U(S) = g_S(U) - \mathrm{acc}(S)$.

Proposition S1. *For every state S, (i) $\Delta_\emptyset(S) = 0$, (ii) $U \subseteq U'$ implies $\Delta_U(S) \le \Delta_{U'}(S)$, and (iii) $0 \le \Delta_U(S) \le 1 - \mathrm{acc}(S)$.*

Proof. (i) The empty coalition has one value combination, so $g_S(\emptyset) = \mathrm{acc}(S)$. (ii) Each value combination x' of U' extends exactly one value combination x of U. For a fixed x,

$$\sum_{x' \supseteq x} \max_c \Pr(h_{U'}{=}x', c \mid S) \;\ge\; \max_c \Pr(h_U{=}x, c \mid S),$$

because the maximum of a sum is at most the sum of the maxima. Summing over x gives $g_S(U') \geq g_S(U)$. (iii) The lower bound follows from (i) and (ii). The upper bound follows from $g_S(U) \leq \sum_x \Pr(h_U = x \mid S) = 1$.

Proposition S2. *For every state S and slot $v \in A_S$, (i) if $\Delta_{U\cup\{v\}}(S) = \Delta_U(S)$ for all $U \subseteq A_S \setminus \{v\}$, then $\phi_v(S) = 0$, (ii) $\sum_{v\in A_S} \phi_v(S) = \Delta_{A_S}(S)$, and (iii) $0 \leq \phi_v(S) \leq \Delta_{A_S}(S)$.*

Proof. The weight $w_{|U|}$ is the probability that U is the set of slots before v in a uniformly random order of A_S. Hence $\phi_v(S)$ is the expected marginal contribution of v over these orders. (i) All marginal contributions of v are zero. (ii) In a fixed order, the marginal contributions of all slots add up to $\Delta_{A_S}(S) - \Delta_\emptyset(S) = \Delta_{A_S}(S)$, and averaging over orders keeps this sum. (iii) Every marginal contribution is non-negative by Proposition S1(ii), so $\phi_v(S) \geq 0$. The upper bound then follows from (ii).

Corollary S1. *Let $k > 0$ and $A_S \neq \emptyset$. The Shapley-greedy policy asks a question if $(1-\varepsilon)\Delta_{A_S}(S) > \lambda|A_S|$, and it submits if $(1-\varepsilon)\Delta_{A_S}(S) \leq \lambda$.*

Proof. By Proposition S2(ii), the largest Shapley value is at least the mean $\Delta_{A_S}(S)/|A_S|$. By Proposition S2(iii), it is at most $\Delta_{A_S}(S)$.

One-step stopping submits when $(1-\varepsilon)\max_v \Delta_{\{v\}}(S) \leq \lambda$. If the slots only help together, every $\Delta_{\{v\}}(S)$ can be close to zero while $\Delta_{A_S}(S)$ is large. Corollary S1 shows that the Shapley-greedy policy keeps asking in such a state whenever the joint gain per slot exceeds the cost. The case study in Section V-F of the main paper is such a state, with a largest one-step gain of 0.055 and a largest Shapley value of 0.230.

A state with two slots shows the effect in its simplest form. Let S contain four interpretations with equal probability, given by the values of two binary slots u and w. The interpretation with $h_u = h_w = 1$ forms result class B, and the other three form class A, so $\mathrm{acc}(S) = 3/4$. An answer to u alone gives $g_S(\{u\}) = \Pr(h_u{=}0, A) + \max\{\Pr(h_u{=}1, A), \Pr(h_u{=}1, B)\} = 1/2 + 1/4 = 3/4$, so $\Delta_{\{u\}}(S) = 0$, and in the same way $\Delta_{\{w\}}(S) = 0$. Answers to both slots identify the class, so $\Delta_{\{u,w\}}(S) = 1/4$. By Eq. (12) of the main paper, $\phi_u(S) = \phi_w(S) = \frac{1}{2}(0-0) + \frac{1}{2}(\frac{1}{4} - 0) = 1/8$. With $\varepsilon = 0.2$ and $\lambda = 0.05$, one-step stopping submits because $(1-\varepsilon)\Delta_{\{u\}}(S) = 0 \leq \lambda$. The Shapley-greedy rule of Eq. (13) asks because $(1-\varepsilon)\phi_u(S) = 0.1 > \lambda$, and the two answers raise the accuracy from $3/4$ to 1.

C. Exactness of Superset Execution

AmbiOverpass obtains the results of all readings from a few superset queries instead of one query per reading. Proposition S3 shows that this returns the same results.

Proposition S3. *Assume that (a) every interpretation h that selects area A has the result $R(h) = \sigma_{\theta_h}(D_A)$ with θ_h of Eq. (7) of the main paper, (b) the truth of t_h, e_h and f_h on an object depends only on its type, its identifier and the attributes in K, and (c) for $r' < r$, $\mathrm{near}_{a,r'}(o)$ implies $\mathrm{near}_{a,r}(o)$, and $\mathrm{around}(X, L_{A,a}, r) = \{o \in X : \mathrm{near}_{a,r}(o)\}$. Then Eq. (10) of the main paper returns $R(h)$ for every interpretation h, and Algorithm 1 of the main paper sends $n_A n_L$ database queries.*

Assumption (a) is Eq. (7) of the main paper. Assumption (b) allows the superset query of Definition 4 and Eq. (8) to fetch each candidate object once and test all target tags, element types and conditions locally. Assumption (c) is the monotonicity of distance in the radius, which lets the nested `around` query of Eq. (9) fetch the largest radius once and recover the smaller radii locally.

Proof. We first show $N_{A,a,r_i} = \{o \in O_A : \mathrm{near}_{a,r_i}(o)\}$ by induction over i. The case $i = 1$ is assumption (c). If the statement holds for r_i, then

$$\begin{aligned} N_{A,a,r_{i+1}} &= \{o \in O_A : \mathrm{near}_{a,r_i}(o) \wedge \mathrm{near}_{a,r_{i+1}}(o)\} \\ &= \{o \in O_A : \mathrm{near}_{a,r_{i+1}}(o)\}, \end{aligned}$$

because $r_{i+1} < r_i$. Next, $t_h \in T$, so θ_h implies $\bigvee_{t\in T} t$ and $\sigma_{\theta_h}(D_A) = \sigma_{\theta_h}(\sigma_{\bigvee_{t\in T} t}(D_A))$. By (b), the projection onto type, identifier and K keeps the truth of $t_h \wedge e_h \wedge f_h$, so selecting these conjuncts from O_A gives $B_A(t_h, e_h, f_h)$. Intersecting it with the sets N_{A,a_j,r_j} applies the proximity conjuncts and gives $R(h)$. One query per pair of a candidate area and a landmark kind computes O_A and all nested sets of Eq. (9), so the algorithm sends $n_A n_L$ queries.

D. Policy Improvement

Theorem 2 of the main paper states that the one-step look-ahead is never worse than the Shapley-greedy policy it builds on.

For a policy π that acts on the state and the remaining budget, let $V_\pi(S, 0) = \mathrm{acc}(S)$ and, for $k > 0$, $V_\pi(S, k) = Q_\pi(S, k, \pi(S, k))$ with

$$\begin{aligned} Q_\pi(S, k, \textsc{Submit}) &= \mathrm{acc}(S), \\ Q_\pi(S, k, v) &= -\lambda + \varepsilon V_\pi(S, k-1) \\ &\quad + (1-\varepsilon)\textstyle\sum_x \Pr(h_v{=}x \mid S) \\ &\quad \cdot V_\pi(S_{v=x}, k-1). \end{aligned}$$

The answers are truthful, and a no-preference answer does not depend on h^*. After any dialogue that reaches S, the posterior of h^* is therefore $p(\cdot \mid S)$. A submission from S is correct with probability $\mathrm{acc}(S)$. A question costs λ and leads to $S_{v=x}$ with probability $(1-\varepsilon)\Pr(h_v = x \mid S)$, or keeps S with probability ε. Induction over k gives $U(\pi) = V_\pi(\mathcal{H}, K)$. The values Q_{sg} and V_{sg} of Eq. (14) of the main paper are Q_π and V_π for $\pi = \pi_{\mathrm{sg}}$.

Theorem 2 (Policy improvement). *Under the prior p, the answer model of Definition 3 of the main paper and the budget K, $U(\pi_{\mathrm{la}}) \geq U(\pi_{\mathrm{sg}})$.*

Proof. Let $V_{\mathrm{la}} = V_{\pi_{\mathrm{la}}}$. We show $V_{\mathrm{la}}(S, k) \geq V_{\mathrm{sg}}(S, k)$ for all S and k by induction over k. For $k = 0$, both policies submit and both values are $\mathrm{acc}(S)$. Assume the claim for

$k-1$. The weights ε and $(1-\varepsilon)\Pr(h_v = x \mid S)$ are non-negative, so $Q_{\mathrm{la}}(S,k,a) \geq Q_{\mathrm{sg}}(S,k,a)$ for every action a, with equality for SUBMIT. The action of π_{sg} is in $\mathcal{A}(S,k)$, and π_{la} maximizes Q_{sg} over this set. Hence

$$\begin{aligned} V_{\mathrm{la}}(S,k) &\geq Q_{\mathrm{sg}}(S,k,\pi_{\mathrm{la}}(S,k)) \\ &\geq Q_{\mathrm{sg}}(S,k,\pi_{\mathrm{sg}}(S,k)) = V_{\mathrm{sg}}(S,k). \end{aligned}$$

Taking $S = \mathcal{H}$ and $k = K$ gives $U(\pi_{\mathrm{la}}) \geq U(\pi_{\mathrm{sg}})$.

E. Cost of One Planning Decision

The following result bounds the work of one planning decision and explains the running times in Section V-G of the main paper.

Proposition S4. *Let $m = |A_S|$, let b be the largest number of values of a slot, and let k be the remaining budget. One decision of π_{la} visits $O(m(b+1)^k)$ states, and an exact search over all question sequences visits $O((m(b+1))^k)$ states. The Shapley values of a visited state S' cost $O(2^m|S'| + 2^m m)$.*

Proof. A question has at most $b+1$ outcomes, b answers and one no-preference answer. The continuation in $Q_{\mathrm{sg}}(S,k,a)$ follows one action of π_{sg} in each later state, so it visits at most $\sum_{i=0}^{k}(b+1)^i = O((b+1)^k)$ states, and one decision evaluates at most $m+1$ actions. An exact search branches over m questions and $b+1$ outcomes in every state, which gives $O((m(b+1))^k)$ states. In a state S', each of the 2^m coalition values takes one pass over S' with a hash table keyed on $(h_U, c(h))$, and each Shapley value sums 2^{m-1} weighted differences.

S5. Implementation Details

Table S4 lists the settings of the simulated user, the LLM components and the baselines. All settings are chosen on the development split and kept fixed on the test and held-out splits.

Table S4
EXPERIMENTAL SETTINGS.

Item	Setting
Simulated user	no-preference probability $\varepsilon = 0.2$, seeds 0–4, same answer stream for all methods
Cost and budget	$\lambda = 0.05$, $K \in \{1,2,3\}$, without a budget up to three interactions per ambiguous slot
LLM	Qwen3-14B-AWQ on vLLM 0.31, thinking disabled
Request weakening	temperature 0.7
Verbalized prior	temperature 0, four option orders averaged, mixed with a uniform distribution of weight 0.05
Sampled prior	12 samples at temperature 1.0, options shuffled per sample, add-0.5 smoothing
Calibration	one power exponent per slot type, fitted on the development split, averaged over 50 bootstrap fits for landmark requests
EC^2-Greedy	EC^2 question choice, asks while $(1-\varepsilon)\max_v \Delta_{\{v\}} > \lambda$
EIG	entropy over readings, submits when one reading reaches probability 0.9
ClarifyGPT	fixed slot order, submits when one result class remains
Generic MCTS	30 simulations per decision, PUCT constant 1.0, uniform action prior, random rollouts, tree keyed by question history, values over readings
LLM planner, ReAct	Qwen3-14B, temperature 0
Significance	paired bootstrap over source queries, 4,000 resamples, 95% CIs

S6. Additional Experimental Analysis

This section reports additional results on the planning subsets and on candidate generation. As in the main paper, EX is the execution accuracy of the submitted query, K is the question budget, #Q is the mean number of questions, and the utility is $\mathrm{EX} - \lambda \#Q$ with $\lambda = 0.05$. SO, SL and DL denote the single-object, single-landmark and dual-landmark requests. The results extend Tables III–V and Fig. 5 of the main paper.

A. Held-Out Splits

This experiment checks whether the results transfer to source queries that played no role in development. The held-out split was built after the planner and the prior were fixed, so no setting was tuned on it. It contains 172 SL requests from 63 source queries and 212 DL requests from 47 source queries. Table IV of the main paper pools these requests with the test split, and Table S5 reports them separately. AmbiOverpass keeps the highest EX at every budget and the highest utility on both subsets. The largest gains appear on DL requests, where the slots interact most.

The gain over EC^2-Greedy comes from slots whose answers help only together. EC^2-Greedy values each question on its own and stops when no single question is worth its cost. On DL requests without a budget, it stops after 4.0 questions. AmbiOverpass credits such slots through their Shapley values, asks 4.5 questions, and raises the utility from 0.701 to 0.768. With a budget of three questions, the first question decides which later questions remain useful. AmbiOverpass scores each first question together with the questions that follow it, which raises EX from 0.483 to 0.539.

The other baselines spend questions that do not change the result. Sphinteract and ClarifyGPT ask the slots in a fixed order. Without a budget they ask 6.6 and 5.2 questions, including slots whose answers leave the result unchanged. EIG prefers questions that split the readings evenly. On DL requests, about three readings return the same objects (Table S8), so part of its questions separate readings with identical results.

A second held-out split was built after all method and prior choices were final, so no setting could be tuned on it. It contains 41 DL requests from 8 source queries (Table S6). With three questions, AmbiOverpass reaches an EX of 0.766, against 0.688 for EC^2-Greedy and 0.712 for EIG.

B. Statistical Significance

This analysis checks which gains on DL requests exceed the variation across source queries. Table S9 reports the

Table S5
RESULTS ON THE LANDMARK SUBSETS OF THE HELD-OUT SPLIT, WITH UTILITY EX$-0.05\cdot$#Q WITHOUT A BUDGET.

Method	Single-landmark				Dual-landmark				
	K=2	K=3	No budget		K=1	K=2	K=3	No budget	
	EX	EX	Utility	#Q	EX	EX	EX	Utility	#Q
No clarification	0.169	0.169	0.169	0.0	0.193	0.193	0.193	0.193	0.0
Sphinteract [3]	0.445	0.656	0.774	4.5	0.208	0.241	0.335	0.669	6.6
ClarifyGPT [4]	0.445	0.656	0.825	3.5	0.208	0.241	0.335	0.739	5.2
EIG [5]	0.535	0.706	0.717	3.8	0.255	0.349	0.478	0.679	5.7
LLM planner	0.407	0.605	0.811	3.8	0.209	0.244	0.338	0.736	5.3
ReAct agent [6]	0.445	0.642	0.807	3.7	0.212	0.235	0.331	0.684	5.2
Generic MCTS [7]	0.501	0.691	0.778	4.1	0.225	0.300	0.406	0.685	6.0
EC^2-Greedy [8]	0.556	0.736	0.823	3.3	0.261	0.349	0.483	0.701	4.0
AmbiOverpass (Ours)	**0.557**	**0.745**	**0.827**	3.2	**0.290**	**0.402**	**0.539**	**0.768**	4.5

Table S6
RESULTS ON THE DUAL-LANDMARK REQUESTS OF THE SECOND HELD-OUT SPLIT.

	EX			No budget	
Method	K=1	K=2	K=3	Utility	#Q
No clarification	0.293	0.293	0.293	0.293	0.0
Sphinteract	0.400	0.478	0.624	0.716	5.7
ClarifyGPT	0.400	0.478	0.624	0.784	4.3
EIG	0.439	0.493	0.712	0.758	4.8
LLM planner	0.351	0.468	0.634	0.786	4.3
ReAct agent	0.351	0.439	0.634	0.734	4.5
Generic MCTS	0.429	0.468	0.620	0.737	5.2
EC^2-Greedy	0.463	0.532	0.688	0.802	3.5
AmbiOverpass (Ours)	**0.478**	**0.561**	**0.766**	**0.820**	3.6

difference between AmbiOverpass and each baseline on the DL requests of the test and held-out splits, with a 95% CI from a paired bootstrap over source queries. The tests separate two sources of the gains. The first source is the result classes. Six clarification baselines do not use them and spend questions on readings that return the same objects. Against these baselines, the CI excludes zero in every cell. The second source is the Shapley valuation with look-ahead. EC^2-Greedy uses the same result classes, so the difference to EC^2-Greedy measures this part alone. This difference also excludes zero in every column, and it grows from 4.5 to 6.5 points with the budget, because a larger budget leaves more states in which several slots help only together.

C. Robustness

This experiment checks whether the ranking depends on the default evaluation settings. AmbiOverpass stays ahead when the evaluation setting changes. Table S7 changes one setting at a time, namely the user seeds, the no-preference probability ε, the question cost λ and the LLM that produces the prior. Each cell gives AmbiOverpass and the best baseline, and AmbiOverpass is higher in every cell. The margin is largest on DL requests at $K = 3$, between 3.2 and 5.7 points. This setting has the most slots that help only together, and the budget forces a choice among them. The prior from Qwen3.5-9B changes the probabilities of the readings but not the result classes, so the execution-based question values stay informative and AmbiOverpass keeps the highest EX in every column.

D. Ambiguity Structure

This analysis describes the planning subsets and relates the gains of AmbiOverpass to their structure. Table S8 shows that the ambiguity grows quickly from SO to DL requests. Readings, classes and state counts are means per instance. The factorisable, gated-forest and general columns count instances by the dependency structure of their slots (Section S1-C), and most instances in every subset are general. From SO to DL requests, the mean number of readings grows from 11.6 to 232.7 and the mean number of result classes from 8.1 to 81.3. On a DL request, about three readings therefore return the same objects. A planner without result classes spends part of its questions on these readings, which contributes to the gap to the six baselines in Table S9.

The last three columns explain why planning stays fast. A DL request has 642,607 nodes in its tree of question histories. Merging histories that reach the same state leaves 1,962 distinct states, and merging states that differ only in the labels of readings and classes leaves 326 canonical states. A planning decision therefore takes 37 to 60 ms (Section V-G of the main paper), and the look-ahead over joint effects stays affordable on the largest requests.

To relate the gains to request complexity, we split the DL requests by the number of ambiguous slots. With three questions, the gain over EC^2-Greedy is 3.2 points (95% CI [0.2, 6.2]) on the 95 requests with at most five slots and 7.0 points ([2.8, 11.2]) on the 187 requests with six or seven slots. More slots create more states in which no single question is worth its cost but several questions together identify the result. EC^2-Greedy stops in these states, while the Shapley stopping rule keeps asking (Corollary S1). On the requests with six or seven slots, the utility gain without a budget reaches 9.7 points ([4.5, 15.7]).

Table S7
RESULTS UNDER DIFFERENT EVALUATION SETTINGS, GIVEN AS AMBIOVERPASS / THE BEST BASELINE.

	SO	SL		DL			
Setting	K=2	K=3	No budget	K=1	K=2	K=3	No budget
Default (ε=0.2, λ=0.05, seeds 0–4)	0.881 / 0.880	0.745 / 0.735	0.820 / 0.816	0.268 / 0.245	0.380 / 0.335	0.529 / 0.472	0.756 / 0.725
Seeds 5–9	0.892 / 0.891	0.732 / 0.732	0.816 / 0.813	0.270 / 0.240	0.379 / 0.335	0.521 / 0.472	0.759 / 0.726
$\varepsilon = 0$	--	0.872 / 0.855	0.856 / 0.853	0.287 / 0.252	0.450 / 0.397	0.649 / 0.592	0.810 / 0.782
$\varepsilon = 0.1$	--	0.819 / 0.801	0.838 / 0.834	0.273 / 0.248	0.413 / 0.365	0.581 / 0.526	0.789 / 0.757
$\varepsilon = 0.3$	--	0.679 / 0.668	0.793 / 0.789	0.254 / 0.230	0.347 / 0.306	0.472 / 0.426	0.723 / 0.688
$\lambda = 0.02$	--	0.747 / 0.738	0.930 / 0.928	0.268 / 0.245	0.384 / 0.338	0.529 / 0.477	0.903 / 0.893
Prior from Qwen3.5-9B	0.852 / 0.845	0.732 / 0.713	0.829 / 0.818	0.241 / 0.211	0.360 / 0.318	0.473 / 0.441	0.746 / 0.725

Table S8
AMBIGUITY STRUCTURE OF THE PLANNING SUBSETS.

		Instances by dependency structure			Mean per instance				
Subset	Instances	Factorisable	Gated forest	General	Readings	Classes	History nodes	Distinct states	Canonical states
Single-object	976	209	70	697	11.6	8.1	75	28	9
Single-landmark	343	82	9	252	45.7	25.1	3,349	196	39
Dual-landmark	435	59	12	364	232.7	81.3	642,607	1,962	326

Table S9
DIFFERENCES BETWEEN AMBIOVERPASS AND EACH BASELINE (POINTS) ON DUAL-LANDMARK REQUESTS, WITH 95% CIS.

	EX		Utility
Baseline	K=2	K=3	No budget
No clarification	+19.9 [14.3, 25.8]	+34.8 [28.2, 41.6]	+57.5 [52.4, 62.3]
Sphinteract	+15.5 [9.5, 21.6]	+21.7 [16.2, 27.2]	+9.6 [8.1, 11.0]
ClarifyGPT	+15.5 [9.5, 21.6]	+21.7 [16.2, 27.2]	+3.1 [2.0, 4.2]
EIG	+4.8 [0.9, 8.8]	+6.5 [3.3, 10.2]	+12.3 [6.7, 19.0]
LLM planner	+15.5 [10.3, 21.0]	+21.2 [16.2, 26.4]	+3.4 [2.2, 4.5]
ReAct agent	+15.5 [9.7, 21.3]	+22.5 [17.2, 28.0]	+8.4 [5.4, 12.6]
Generic MCTS	+9.9 [6.2, 13.7]	+14.3 [10.9, 17.8]	+7.7 [6.5, 9.1]
EC^2-Greedy	+4.5 [1.1, 7.9]	+5.7 [2.7, 8.9]	+6.5 [2.7, 10.8]

Table S10
COMPONENT ABLATION ON THE SINGLE-LANDMARK SUBSET.

Configuration	EX		
	K=1	K=2	K=3
w/o validity pruning	0.115	0.292	0.499
w/o result classes	0.323	0.502	0.683
w/o LLM prior	0.259	0.459	0.652
Full system	**0.343**	**0.572**	**0.745**

E. Ablation on Single-Landmark Requests

This ablation measures what each component contributes on the 234 SL requests, which have fewer slots than DL requests and complement the DL ablation in Table V of the main paper. Each row of Table S10 disables one component. Removing validity pruning costs 22.8 to 28.0 points, because questions and probability then go to readings that return no objects. Removing result classes costs 2.0 to 7.0 points, because questions that only separate readings with the same objects then count as useful. Replacing the LLM prior with a uniform prior costs 8.4 to 11.3 points. With one to three questions, some slots stay unasked, and the submitted class then depends on which readings the request text makes likely.

F. Candidate Generation

Table S11
COVERAGE OF THE INTENDED TAG ON THE REWRITING SUBSET BEFORE AND AFTER QUALITY CONTROL (QC), WITH THE MEAN NUMBER OF CANDIDATE TAGS.

	Coverage (%)		
Candidate source	All (273)	QC (254)	Tags
Precise request, 8 tags (reference)	74.0	73.2	$\leq$8
One LLM call, 8 tags	44.3	45.3	$\leq$8
One LLM call, 50 tags	50.5	52.8	38.9
Repeated sampling	58.2	59.8	23.5
Paraphrasing	57.5	59.4	25.3
Rewriting operations	**67.0**	**69.7**	31.8

Table S12
COVERAGE OF THE INTENDED TAG BY PERTURBATION TYPE.

		Coverage (%)	
Perturbation	Requests	One call, 50 tags	Rewriting
Everyday wording	148	50.7	**66.2**
Unrecorded property	125	50.4	**68.0**
Uncheckable requirement	207	51.7	**68.6**
Bare place name	195	48.7	**66.7**

This experiment extends Section V-D of the main paper and checks whether the rewriting operations recover intended tags that a single LLM call misses. Table S11 reports how often the intended tag is among the candidate tags, on the 273 syntax-valid rewriting requests and on the 254 requests that pass quality control. On the 254 requests, rewriting reaches 69.7%, against 52.8% for a single list of 50 tags. Repeated sampling and paraphrasing use as many LLM calls as rewriting and reach about 59%, so the gain comes from the operations and not from the extra calls. For reference, aligning the original precise request reaches 74.0%.

The gain holds for every perturbation type, where rewriting adds 15.5 to 18.0 points (Table S12). Correction replaces an unrecorded property with recordable kinds of objects, and relaxation removes requirements that the database cannot check. Each rewritten request then names a concrete kind of object, which tag alignment maps to OSM tags.

S7. Prompts

This section lists the prompts for benchmark construction, the interpretation prior, candidate generation and the LLM baselines. All prompts go to Qwen3-14B with thinking disabled (Table S4). Text in braces is filled in for each request. Prompts S1–S5 build the benchmark (Section S1), Prompts S6 and S7 give the interpretation prior (Section IV-A of the main paper), Prompts S8 and S9 generate candidate tags (Section V-D of the main paper), and Prompts S10 and S11 are the LLM baselines.

A. Benchmark Construction

Prompt S1 weakens a precise single-object request. Its user message lists one instruction per weakened slot, and the instructions are given below the template. Prompts S2 and S3 add one or two proximity constraints to a precise request and write the matching ambiguous request. Prompt S4 produces the rewriting subset, and Prompt S5 is the recoverability check of the quality control.

Prompt S1. Weakening a single-object request (temperature 0.7).

```
[System]
You simulate a non-expert user asking for map data.
You get a precise request and a list of things to make
     vague. Produce ONE rewritten request that a real
     person might type.

Hard rules:
- Same subject as the precise request. Examples in the
     instructions show the STYLE of change only.
- Make vague ONLY the listed parts; everything else stays
     exactly as specific as before.
- Never mention OSM tags, keys, or query syntax (no
     'amenity=', no 'tagged as', no 'node', no 'way', no
     'relation'), never copy ids or coordinate lists.
- One sentence, natural and casual. Output only the
     rewritten request.

[User]
Precise request: {precise request}
Make vague:
- {instruction for slot 1}
- {instruction for slot 2}
Rewritten request:

[Instructions]
area:      Mention the place only by its bare name,
     dropping any country, region or 'city of' qualifier
     that would tell it apart from other places with the
     same name.
tag:       Describe the KIND OF OBJECT ({key = value}) in
     vague everyday words that could fit several related
     kinds of objects, e.g. 'charging stations' ->
     'places to power up my car'.
type:      Make it unclear whether the user wants points,
     outlines/areas or larger groupings: drop words that
     signal the geometry (e.g. 'buildings' -> 'these
     places', 'roads' -> 'them').
condition: Silently leave out this one condition:
     {condition}. Do not signal that anything is missing.
```

Prompt S2. Single-landmark request.

```
A precise map request: "{precise request}"
Extend it so that it only asks for these objects within
     {r} metres of {landmark} ({landmark tag}), then
     write how a non-expert would type it. In the vague
     version: describe the main objects in vague everyday
     words that could fit several related kinds; replace
     the exact distance by "near" or "close to"; name the
     anchor in everyday words that could fit several
     kinds of {landmark kind}; keep the place name. Never
     mention OpenStreetMap tags or query syntax.
Output JSON only: {"precise": "...", "vague": "..."}
```

Prompt S3. Dual-landmark request.

```
A precise map request: "{precise request}"
Extend it so that it only asks for these objects that are
     within {r1} metres of {landmark 1} ({tag 1}) and
     also within {r2} metres of {landmark 2} ({tag 2}),
     then write how a non-expert would type it. In the
     vague version: describe the main objects in vague
     everyday words that could fit several related kinds;
     replace both exact distances by "near" or "close
     to"; name both landmarks in everyday words that
     could fit several kinds; keep the place name. Never
     mention OpenStreetMap tags or query syntax.
Output JSON only: {"precise": "...", "vague": "..."}
```

Prompt S4. Rewriting subset. The object is always perturbed by one of the first two instructions, together with one or two of the last two.

```
[System]
You simulate a non-expert user asking for map data.
You get a precise request and instructions for how to
     make it harder to interpret. Produce ONE rewritten
     request that a real person might type.
Rules: same subject; follow every instruction and change
     nothing else; never mention OpenStreetMap tags, keys
     or query syntax; one natural sentence. Output only
     the rewritten request.

[Instructions]
unrecorded property:     Describe the main kind of object
     ({object}) through a property OpenStreetMap does not
     record, one step off from what it really is (style
     examples: 'the best restaurants', 'five-star
     hotels', 'new buildings'). The real kind of object
     must still be inferable, but do not name it.
everyday wording:        Replace the name of the main
     kind of object ({object}) with vague everyday words
     that could fit several related kinds of objects. Do
     not use the original word.
uncheckable requirement: Add one natural-sounding
     requirement that a map database cannot express, such
     as being quiet, popular, cheap, nice or
     well-reviewed.
bare place name:         Refer to the place only by its
     bare name, without country, region or administrative
     words.
```

Prompt S5. Recoverability check in quality control (temperature 0).

```
Request: "{rewritten request}"
Could a careful reader work out that the user mainly
     wants {object} (OpenStreetMap {tag}) (it may be one
     of a few plausible readings)? Answer only yes or no.
```

B. Interpretation Prior

Prompt S6 asks for a probability for each candidate value of a slot. It is issued with four orders of the options, and the four distributions are averaged. Prompt S7 gives the sampled-choice estimate for the target tag and for the element type of single-object requests. The options are shuffled in each of 12 samples at temperature 1.0, and the counts of the chosen options give the prior.

Prompt S6. Verbalized prior (temperature 0).

```
A user typed this map search: "{ambiguous request}"
For the question of {slot description}, how likely is
     each option to be what the user means? Options:
1. {value 1}
2. {value 2}
...
Give a probability for every option (they should sum to
     1). Output JSON only: a list of numbers in option
     order.

[Slot descriptions]
area:      which place is meant
tag:       which kind of map object is meant
type:      which geometry is meant (node = point, way =
     line or outline, relation = grouped object, nwr =
     any)
condition: whether this extra condition is meant
landmark:  which kind of nearby landmark is meant
           (two landmarks: which kind of the first /
     second nearby landmark is meant)
radius:    which search radius in metres is meant
           (two landmarks: which radius around the first
     / second landmark is meant)
```

Prompt S7. Sampled-choice prior (temperature 1.0, 12 samples). For the element type, the question line reads "Which geometry does the user mean (node = point, way = line or outline, relation = grouped object, nwr = any)?"

```
A user typed this map search: "{ambiguous request}"
Which kind of map object does the user mean? Options:
1. {value in shuffled position 1}
2. {value in shuffled position 2}
...
Answer with the number of one option only.
```

C. Candidate Generation

Prompt S8 aligns a request to OSM tags. Prompt S9 applies correction and relaxation and returns four precise requests, and each of them is then aligned to ten tags.

Prompt S8. Tag alignment.

```
[System]
You map natural-language map requests to OpenStreetMap.
     Answer with JSON only.
[User]
Request: {request}
List {n} different OpenStreetMap tags (key=value) that
     could describe the MAIN kind of object the user
     wants, most likely first. Output a JSON list of
     strings like ["amenity=cafe", ...].
```

Prompt S9. Correction and relaxation (temperature 0.5).

```
[System]
You map natural-language map requests to OpenStreetMap.
     Answer with JSON only.
[User]
Request: {request}
The request may describe objects through properties a map
     database does not record (e.g. 'best', 'five-star',
     'new') or add requirements it cannot check (e.g.
     'quiet', 'cheap'). Write 4 alternative precise
     versions of the request that a map database can
     answer: replace unrecordable properties with the
     closest recordable kinds of objects and drop
     requirements it cannot check. Each version should
     commit to a different plausible kind of object.
     Output a JSON list of 4 strings.
```

D. LLM Baselines

The LLM planner (Prompt S10) and the ReAct agent (Prompt S11) see the same open slots and remaining values as the other planners. The ReAct agent also sees the dialogue so far and the numbers of remaining readings and result classes.

Prompt S10. LLM planner (temperature 0).

```
A user asked a map database: "{ambiguous request}"
Some parts of the request are unclear. You may ask the
     user at most {k} more question(s), each about ONE of
     these parts, and every question costs the user
     effort:
- {slot}: {slot description}. Options still possible:
     {value 1}; {value 2}; ...
...
Which part should be asked about next? If asking is not
     worth it, answer STOP. Answer with exactly one word:
     one of [{slot names}] or STOP.
```

Prompt S11. ReAct agent (temperature 0).

```
You help a user query a map database. The user asked:
     "{ambiguous request}"
Dialogue so far:
Q: {slot description}? -> A: {answer or "no preference"}
...
Observation: {n} interpretations of the request are still
     possible; executed on the database they give {m}
     different results. If you stop now, the system
     returns the most likely result.
Open parts you may ask about (each question costs the
     user effort; at most {k} more question(s)):
- {slot}: {slot description}. Options still possible:
     {value 1}; {value 2}; ...
...
Think step by step in at most three sentences about which
     part, if any, is worth asking, then give your action.
Format:
Thought: ...
Action: ASK <part> or Action: STOP, where <part> is one
     of [{slot names}].
```

REFERENCES

[1] M. Staniek *et al.*, "Text-to-overpassql: A natural language interface for complex geodata querying of openstreetmap," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 562–575, 2024.

[2] H. Laurent and R. L. Rivest, "Constructing optimal binary decision trees is np-complete," *Information processing letters*, vol. 5, no. 1, pp. 15–17, 1976.

[3] F. Zhao *et al.*, "Sphinteract: Resolving ambiguities in nl2sql through user interaction," *Proceedings of the VLDB Endowment*, vol. 18, no. 4, pp. 1145–1158, 2024.

[4] F. Mu *et al.*, "ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2332–2354, 2024.

[5] L. Qiu *et al.*, "Interactive text-to-sql via expected information gain for disambiguation," *arXiv preprint arXiv:2507.06467*, 2025.

[6] S. Yao *et al.*, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[7] W. Xu *et al.*, "VisInteract: Towards dynamic interactive text-to-visualization under imperfect queries," in *Advances in Neural Information Processing Systems*, 2026.

[8] D. Golovin *et al.*, "Near-optimal bayesian active learning with noisy observations," *Advances in Neural Information Processing Systems*, vol. 23, 2010.